

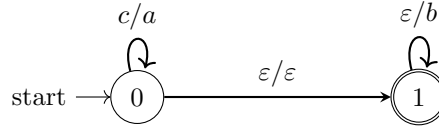


PAUTA INTERROGACIÓN 2

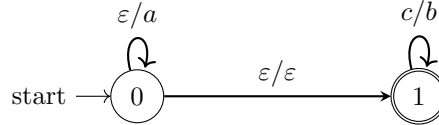
Pregunta 1

Si la afirmación se cumpliera para todo transductor, debería cumplirse para los siguientes:

- Se construye $\mathcal{T}_1$ de la siguiente manera:



- Se construye $\mathcal{T}_2$ de la siguiente manera:



Es fácil ver que $\llbracket \mathcal{T}_1 \rrbracket = \{(c^n, a^n b^m) \mid n \geq 0, m \geq 0\}$ y $\llbracket \mathcal{T}_2 \rrbracket = \{(c^n, a^m b^n) \mid n \geq 0, m \geq 0\}$.

Luego, usando la intersección de transductores, $\llbracket \mathcal{T}_1 \rrbracket \cap \llbracket \mathcal{T}_2 \rrbracket = \{(c^n, a^n b^n) \mid n \geq 0\}$.

Calculando $\pi_2(\llbracket \mathcal{T}_1 \rrbracket \cap \llbracket \mathcal{T}_2 \rrbracket) = a^n b^n$, podemos ver que el resultado no es un lenguaje regular (demostrado en clases).

Finalmente, sabemos que para todo transductor $\mathcal{T}$, se debe cumplir que $\pi_2(\llbracket \mathcal{T} \rrbracket)$ es regular, por lo tanto, $\llbracket \mathcal{T}_1 \rrbracket \cap \llbracket \mathcal{T}_2 \rrbracket$ no puede ser una relación racional.

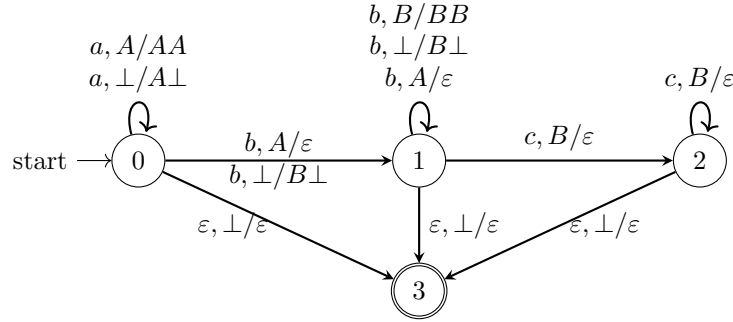
Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por encontrar dos transductores que sirvan para dar un contraejemplo. Para cada transductor se define el siguiente puntaje:
 - **(1 punto)** Por hacer la construcción del transductor.
 - **(0.5 puntos)** Por definir la función $\llbracket \mathcal{T} \rrbracket$ del transductor.
- **(1 punto)** Por dar la función de la intersección $\llbracket \mathcal{T}_1 \rrbracket \cap \llbracket \mathcal{T}_2 \rrbracket$
- **(1 punto)** Por decir que el lenguaje de output del transductor $a^n b^n$ no es regular (análogamente se pudo haber encontrado para el input)
- **(1 punto)** Por explicar por qué el lenguaje $a^n b^n$ no es regular.

Pregunta 2

L_2 es un lenguaje libre de contexto y su PDA o PDA alternativo es uno de los siguientes:

PDA



PDA alternativo

$$D = (Q, \Sigma, \Delta, q_0, F)$$

- $Q = \{q_0, q_f, A, B\}$
- $q_0 = q_0$
- $F = \{q_f\}$
- $\Delta = \{(q_0, a, Aq_f), (A, a, AA), (A, b, \varepsilon), (q_f, b, Bq_f), (B, b, BB), (B, c, \varepsilon), (q_0, \varepsilon, q_f), (q_0, b, Bq_f)\}$

La idea de ambos autómatas es comenzar haciendo push de las i letras a , manteniéndose en el estado 0. Luego, al leer la primera letra b , se comenzará a hacer pop, en el estado 1, hasta que no queden letras A en el stack, para luego hacer push de cada b restante. Finalmente, al leer la primera letra c , se pasará al estado 2 para hacer pop de todas las B agregadas anteriormente y se llegará al estado final 3 cuando ya no queden letras B en el stack.

Si la cantidad de letras b es menor a la de a o la cantidad de letras c es menor a la diferencia de b y a , no se llegará nunca a estado final y, similarmente, si alguna de las cantidades es mayor a la deseada, se llegará al estado final, pero no se terminará de leer la palabra.

L_1 no es un lenguaje libre de contexto, para demostrarlo se usará el lema de bombeo.

Para todo $N > 0$, tomamos la palabra $z = a^N b^{N^2} c^N \in L_1$ con $|z| \geq N$. Sea $z = uvwx$ la descomposición de z , que se divide en los siguientes casos (considerando $vx \neq \varepsilon$ y $|vwx| \leq N$):

- $vwx = a^N = a^l a^j a^k$, luego $v^i w x^i = a^{li} a^j a^{ki}$, entonces si tomamos $i = 2$, nos queda $v^i w x^i = a^{2l} a^j a^{2k} = a^{N+j+k}$, lo cual hace que $a^{N+j+k} b^{N^2} c^N \notin L_1$.
Los casos $vwx = b^N$ y $vwx = c^N$ se resuelven análogamente.
- $vwx = a^j b^k$, luego salen 3 casos:
 - $(v = a^l \wedge x = \varepsilon) \vee (v = \varepsilon \wedge x = b^m)$ se resuelve de manera análoga al ítem anterior.
 - $v = a^l, x = b^m$, entonces si tomamos $i = 0$, nos queda $a^{N-l} b^{N^2-m} c^N$, lo cual pertenece a L_1 ssi $N^2 - m = N(N-l)$ y $N(N-l) = N^2 - Nl$, despejando llegamos a que $m = Nl$ lo cual es una contradicción, ya que asumimos que $|vwx| \leq N$, entonces $a^{N-l} b^{N^2-m} c^N \notin L_1$.

- $v = a^l b^m \vee x = a^l b^m$, entonces si tomamos $i = 0$, nos queda $a^{N-l} b^{N^2-m} c^N$, lo cual se resuelve de manera análoga al ítem anterior.
- $vw x = b^j c^k$, se resuelve de manera análoga al ítem anterior.

Como se aplicó lema de bombeo y se estudiaron los distintos casos, se concluye que L_1 no es un lenguaje libre de contexto.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por demostrar correctamente L_2 es un lenguaje libre de contexto con su PDA correspondiente.
 - **(1.5 puntos)** Por realizar correctamente el PDA.
 - **(0.5 puntos)** Por realizar correctamente push y pop de letras a y c .
 - **(0.5 puntos)** Por realizar correctamente el pop y luego push de la letra b .
 - **(0.5 puntos)** Por realizar correctamente la condición de término.
 - **(1.5 puntos)** Por explicar correctamente el PDA.
 - **(0.5 puntos)** Por explicar correctamente push y pop de letras a y c .
 - **(0.5 puntos)** Por explicar correctamente el pop y luego push de la letra b .
 - **(0.5 puntos)** Por explicar correctamente la condición de término.
- **(3 puntos)** Por demostrar correctamente L_1 no es un lenguaje libre de contexto con el lema de bombeo.
 - **(0.5 puntos)** Por elegir una palabra a la que se le puede aplicar el lema.
 - **(0.5 puntos)** Por demostrar correctamente los casos $vw x = a^N \vee b^N \vee c^N$.
 - **(0.5 puntos)** Por demostrar correctamente los casos $v = a^l, x = b^m$.
 - **(0.5 puntos)** Por demostrar correctamente los casos $v = a^l b^m \vee x = a^l b^m$.
 - **(0.5 puntos)** Por demostrar correctamente los casos $vw x = b^j c^k$.
 - **(0.5 puntos)** Por llegar correctamente a la contradicción $m = Nl$.
 - **(-0.5 puntos)** Por no mostrar correctamente el caso $vw x = a^j b^k \longrightarrow (v = a^l \wedge x = \varepsilon) \wedge (v = \varepsilon \wedge x = b^m)$.

Pregunta 3

Dada una gramática libre de contexto $\mathcal{G} = (V, \Sigma, P, S)$ en forma normal extendida de Chomsky, construiremos una nueva gramática $\mathcal{G}' = (V^0 \cup V^1, \Sigma, P', S^0)$ que solamente genera las palabras que tengan un árbol de derivación en G con una cantidad par de nodos, donde:

1. Se define V^α , donde $\alpha \in \{0, 1\}$, como:

$$V^\alpha = \{X^\alpha \mid X \in V\}$$

Con esto, la nueva gramática tendrá dos versiones de cada variable en V : una versión X^0 que generará sub-árboles con cantidad par de nodos y una versión X^1 que generará sub-árboles con una cantidad impar de nodos.

2. Construimos P' como:

- Si $X \rightarrow a \in P$, entonces se añade:

$$X^0 \rightarrow a \in P'$$

Esto se debe a que la derivación tiene exactamente 2 nodos (la variable y el terminal), lo cual genera un árbol par.

- Si $X \rightarrow Y_1 \dots Y_k \in P$, entonces se añaden las siguientes producciones a P' :

$$X^0 \rightarrow Y_1^{p_1} \dots Y_k^{p_k}, \text{ tal que } Y_j^{p_j} \in P', \text{ y } \sum_{i=1}^k p_i \equiv 1 \pmod{2}, \forall p_i \in \{0, 1\}$$

$$X^1 \rightarrow Y_1^{p_1} \dots Y_k^{p_k}, \text{ tal que } Y_j^{p_j} \in P', \text{ y } \sum_{i=1}^k p_i \equiv 0 \pmod{2}, \forall p_i \in \{0, 1\}$$

La primera línea garantiza que la derivación desde X^0 tenga un número impar de nodos para toda combinación de sub-derivaciones. Mientras que la segunda línea asegura un número par de nodos para la derivación desde X^1 .

Definimos la variable inicial como S^0 para hacer que $\mathcal{G}'$ solo pueda generar las palabras que tienen árboles de derivación con una cantidad par de nodos. Con esta construcción, $\mathcal{G}'$ logra distinguir a las derivaciones pares y por ende definir a $L_2(\mathcal{G})$.

Para ayudar con la notación, definimos $|t|$ como el número de nodos del árbol de derivación t y para toda palabra $w \in \Sigma^*$, diremos que $t(w)$ es un árbol de derivación cualquiera, formado por producciones de $\mathcal{G}$, tal que $\text{yield}(t(w)) = w$ y que $t'(w)$ es un árbol de derivación cualquiera, formado por producciones de $\mathcal{G}$, tal que $\text{yield}(t'(w)) = w$. Cabe señalar que $t(w)$ y $t'(w)$ pueden no existir para una palabra $w \in \Sigma^*$.

Buscamos demostrar que:

$$w \in L(\mathcal{G}) \wedge \exists t. |t(w)| \text{ es par} \Leftrightarrow w \in L(\mathcal{G}').$$

A continuación haremos una demostración informal de lo anterior.

Usando inducción estructural, demostraremos que para toda palabra $w \in \Sigma^*$ se cumple que:

$$\begin{aligned} \exists |t(w)| \wedge |t(w)| \text{ es par} &\Leftrightarrow \exists t'(w) \wedge \text{raiz}(t') \in V^0 \wedge \\ \exists |t(w)| \wedge |t(w)| \text{ es impar} &\Leftrightarrow \exists t'(w) \wedge \text{raiz}(t') \in V^1 \end{aligned}$$

■ **CB:**

($\Rightarrow$) El árbol de derivación más pequeño que podemos crear con las variables de $\mathcal{G}$ tiene dos nodos, proviene de la producción $X \rightarrow a \in P$ y tiene la forma:

$$\begin{array}{c} X \\ | \\ a \end{array}$$

Por la definición de $\mathcal{G}'$, si $X \rightarrow a \in P$, entonces $X^0 \rightarrow a \in P'$, por lo que podemos generar el árbol:

$$\begin{array}{c} X^0 \\ | \\ a \end{array}$$

($\Leftarrow$) Análogo al caso anterior, el árbol de derivación más pequeño que podemos crear con las variables de $\mathcal{G}'$, proviene de la producción $X^0 \rightarrow a \in P'$ y tiene la forma:

$$\begin{array}{c} X^0 \\ | \\ a \end{array}$$

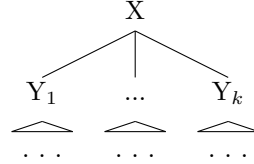
Por la definición de $\mathcal{G}'$, si $X^0 \rightarrow a \in P'$, entonces $X \rightarrow a \in P$, por lo que podemos generar el árbol:

$$\begin{array}{c} X \\ | \\ a \end{array}$$

que tiene dos nodos.

- **HI:** Para todo árbol de derivación $t(w)$, en $\mathcal{G}$, tal que $|t(w)| < n$, existe $t'(w)$ en $\mathcal{G}'$ tal que $\text{raiz}(t') \in V^0$ si $|t(w)|$ es par y $\text{raiz}(t') \in V^1$ si $|t(w)|$ es impar. Además, para todo árbol de derivación $t'(w)$, en $\mathcal{G}'$, tal que $|t'(w)| < n$, existe $t(w)$ en $\mathcal{G}$ tal que $|t(w)|$ es par si $\text{raiz}(t') \in V^0$ y $|t(w)|$ es impar si $\text{raiz}(t') \in V^1$.
- **PI:**

($\Rightarrow$) Sea t un árbol de derivación de tamaño n , como $\mathcal{G}$ está en extended-CNF, sabemos que el árbol debe tener la siguiente forma:

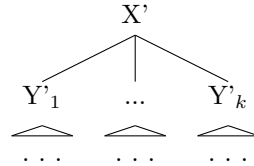


Por hipótesis de inducción, sabemos que existe un árbol de derivación en $\mathcal{G}'$ para cada una de las palabras generadas por Y_1 hasta Y_k , por lo que dependiendo del tamaño de estos árboles en $\mathcal{G}$, podemos distinguir dos casos generales:

1. *La sumatoria de los tamaños de todos los árboles cuya raíz tiene como antecesor a X^1 tiene tamaño par:* En este caso, nuestro árbol con raíz en X^1 tendrá una cantidad impar de nodos. Por hipótesis de inducción, tendremos árboles en $\mathcal{G}'$ que tienen como raíz a $Y_1^{p_1}$ hasta $Y_k^{p_k}$, es decir tendremos combinaciones de árboles donde se cumpla que la sumatoria del número de los nodos de cada árbol es par. Usando la producción $X^1 \rightarrow Y_1^{p_1} \dots Y_k^{p_k}$, podemos generar un árbol en $\mathcal{G}'$ con raíz en $X^1 \in V^i$.
2. *La sumatoria de los tamaños de todos los árboles cuya raíz tiene como antecesor a X^0 tiene tamaño impar:* En este caso, nuestro árbol con raíz en X^0 tendrá una cantidad par de nodos. Por hipótesis de inducción, tendremos árboles en $\mathcal{G}'$ que tienen como raíz a $Y_1^{p_1}$ hasta $Y_k^{p_k}$, es decir tendremos combinaciones de árboles donde se cumpla que la sumatoria del número de los nodos de cada árbol es impar. Usando la producción $X^0 \rightarrow Y_1^{p_1} \dots Y_k^{p_k}$, podemos generar un árbol en $\mathcal{G}'$ con raíz en $X^0 \in V^i$.

Dado que cubrimos todos los posibles árboles de tamaño n , hemos demostrado la propiedad deseada para todo árbol.

($\Leftarrow$) Sea t' un árbol de derivación de tamaño n . Como $\mathcal{G}'$ está en extended-CNF, sabemos que el árbol debe tener la siguiente forma:



Debido a como se construyó $\mathcal{G}'$, podemos replicar todos sus árboles en $\mathcal{G}$, removiendo sus *labels*, por lo que solo necesitamos demostrar que el tamaño de estos árboles es el correcto. Es claro que el tamaño del árbol estará dado por $|t'_X| = |t_{Y'_1}| + \dots + |t_{Y'_k}| + 1$, por lo que dependiendo del *label* de la variable X' tendremos dos casos generales con $j \in \mathbb{N}$:

1. $X' \in V^1$: Dadas las producciones de $\mathcal{G}'$, por hipótesis de inducción, $|t_{Y'_1}| + \dots + |t_{Y'_k}| = 2j$, por lo que $|X'| = 2j + 1$, lo que corresponde a un número impar.

2. $X' \in V^0$: Dadas las producciones de $\mathcal{G}'$, por hipótesis de inducción, $|t_{Y'_1}| + \dots + |t_{Y'_k}| = 2j + 1$, por lo que $|X'| = 2j + 2$, lo que corresponde a un número par.

Dado que cubrimos todos los posibles árboles de tamaño n , hemos demostrado la propiedad deseada para todo árbol.

Finalmente, podemos tomar el caso en que tenemos un árbol de derivación de $\mathcal{G}$ con raíz en S^0 y cantidad par de nodos. Es claro que este es un caso particular de la propiedad demostrada y también que es equivalente a lo que queríamos demostrar, por tanto:

$$w \in L(\mathcal{G}) \wedge \exists t. |t(w)| \text{ es par} \Leftrightarrow w \in L(\mathcal{G}')$$

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por entregar correctamente la construcción de $\mathcal{G}'$.
 - **(0.5 puntos)** Por definir correctamente V' .
 - **(0.5 puntos)** Por definir correctamente S^0 .
 - **(0.5 puntos)** Por definir correctamente $X \rightarrow a \in P$, entonces $X^0 \rightarrow a \in P$
 - **(1.5 puntos)** Por definir correctamente los dos casos de $X \rightarrow Y_1 \dots Y_k \in P$.
- **(3 puntos)** Por entregar una explicación correcta que $\mathcal{G}'$ define el lenguaje pedido.
 - **(1 punto)** Por explicar correctamente el caso base de la demostración.
 - **(1 punto)** Por explicar correctamente que si $w \in L(\mathcal{G}) \wedge \exists t. |t(w)| \text{ es par} \Rightarrow w \in L(\mathcal{G}')$.
 - **(1 punto)** Por explicar correctamente que si $w \in L(\mathcal{G}') \Rightarrow w \in L(\mathcal{G}) \wedge \exists t. |t(w)| \text{ es par}$.

Pregunta 4

Algoritmo

Para un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ y una palabra $w = a_1 a_2 \dots a_n$, tenemos:

```

Function eval-#DFAonthefly ( $\mathcal{A}, w$ )
   $S_{old} := I$ 
   $S_{older} := \emptyset$ 
  for  $i = 1$  to  $n$  do
     $S_{older} := S_{old}$ 
     $S_{old} := S$ 
     $S := \emptyset$ 
    foreach  $p \in S_{older}$  do
       $S_{old} := S_{old} \cup \{q \mid (p, a_i, q) \in \Delta\}$ 
       $S := S \cup \{q \mid (p, a_i a_{i+1}, q) \in \Delta\}$ 
    end
  end
  return  $check(S_{old} \cap F \neq \emptyset)$ 
end

```

Explicación

El algoritmo es similar al **NFA on-the-fly** con la modificación de que se incluye un S_{older} . Con lo anterior ahora se diferencian los estados alcanzados con la lectura de una letra con los de dos letras. Por cada iteración por posición de la palabra se actualizan los conjuntos S_{older}, S_{old} con los valores de S_{old}, S , respectivamente para denotar que se avanzó una letra, además se vacía el conjunto S para luego actualizarlo con las letras de la respectiva iteración.

Se itera por S_{older} para obtener las letras a las que se ha llegado y se almacenan en S_{old} si son alcanzables con transición que lee una letra, y en S para las dos letras.

Finalmente, se revisan las letras alcanzables por el algoritmo revisando la intersección entre S_{old} y los estados finales del autómata

Complejidad

Explicación operaciones en algoritmo

- No se usa la transición $(p, a_i a_{i+1}, q) \in \Delta$ en el caso en el que $i + 1 > n$, ya que no queda por leer en la palabra
- Para las transiciones del tipo $(p, u_j, q) \in \Delta$, con $j \in \Sigma \cup \Sigma^2$, se tienen de tal forma que solo se iteran por las transiciones que parten en p estado que está definido antes de iterar por Δ , de esta forma, solo se revisan las transiciones de p

Complejidad operaciones en algoritmo

- El algoritmo tiene tres conjuntos S, S_{old}, S_{older} , ya que tienen estados del autómata, el tamaño para todos ellos está acotado por $|Q|$
- El algoritmo tiene tres loops anidados, con el tercero teniendo dos loops por Δ . El primero itera por el largo de la palabra $\mathcal{O}(|w|)$, el segundo itera por los estados en S_{old} ($\mathcal{O}(Q)$), el tercero itera por las transiciones de p , por la forma que está definido, sólo itera por las transiciones que parten en p , y lo hace dos veces para revisar las transiciones que tienen una y dos letras.
- Por lo explicado en el item anterior, la operación de los dos loops anidados que iteran por Q y por Δ es $\mathcal{O}(|Q| + |\Delta|)$
- Tanto las uniones, intersecciones y actualizaciones de los estados en los conjuntos $S, S_{old}, S_{older}, F, I$, se hacen en $\mathcal{O}(Q)$ al todos estar acotados por la cantidad de estados del autómata $|Q|$

Con lo anterior, se tiene que la complejidad del algoritmo es

$$\mathcal{O}(|Q| + |w| \cdot (|Q| + |\Delta|))$$

Ya que $\mathcal{A} = |Q| + |\Delta|$, se tiene entonces que la complejidad es:

$$\mathcal{O}(|w| \cdot |\mathcal{A}|)$$

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(3 puntos)** Por Algoritmo
 - **(3 puntos)** Algoritmo correcto, con explicación y sin errores
 - **(2.5 puntos)** Algoritmo correcto, con explicación, pero con un error menor

- **(2.0 puntos)** Algoritmo correcto, con explicación, pero con a lo más dos errores menores
 - **(1.5 puntos)** Algoritmo correcto, sin explicación y sin errores
 - **(0 puntos)** En otro caso
- **(1.5 puntos)** Por Explicación
 - **(0.5 puntos)** Explica NFA mencionando la estructura que guarda los estados y cómo los va guardando
 - **(0.5 puntos)** Explica cambios a NFA para que funcione el algoritmo. Rige si es que el algoritmo no es exponencial
 - **(0.5 puntos)** Los cambios a NFA explicados funcionan para el problema y el algoritmo presenta esos cambios correctamente. Rige si es que el algoritmo no es exponencial
 - **(1.5 puntos)** Por Complejidad
 - **(0.5 puntos)** Explica la complejidad final del algoritmo correctamente mencionando que se itera por la palabra y por el autómata (Q y Δ).
 - **(0.5 puntos)** Explica por qué se usa $|\Delta| + |Q|$ en vez de $|\Delta| \cdot |Q|$
 - **(0.5 puntos)** Explica en detalle la complejidad de los cambios realizados al NFA on the fly. Rige si es que el algoritmo no es exponencial.

Anexo: Algoritmo que usa tuplas

Se deja este anexo para la cantidad de personas que usaron tuplas, para que tengan un ejemplo de cómo se hacía para ese caso

Para un DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ y una palabra $w = a_1 a_2 \dots a_n$, tenemos:

```

Function eval-#DFAonthefly ( $\mathcal{A}, w$ )
   $S := \{(q, 1) \mid q \in I\}$ 
  for  $i = 1$  to  $n$  do
     $S_{old} := S$ 
     $S := \emptyset$ 
    foreach  $(p, j) \in S_{old}$  do
      if  $j \neq i$  then
         $S := S \cup \{(p, j)\}$ 
      end
      else
         $S := S \cup \{(q, i + 1) \mid (p, a_i, q) \in \Delta\}$ 
         $S := S \cup \{(q, i + 2) \mid (p, a_i a_{i+1}, q) \in \Delta\}$ 
      end
    end
  end
   $S_Q := \{q \mid (q, j) \in S \wedge j = n + 1\}$ 
  return  $check(S_Q \cap F \neq \emptyset)$ 
end

```

Explicación a grandes rasgos:

- Definimos S y el **foreach** para que admitan las tuplas
- El $j \neq i$ permite no avanzar de más en el S , controlando así su tamaño para que solo contenga a lo más $|Q| \cdot 2$ elementos (tanto para S como S_{old})

- Notar que si no hacemos el $j \neq i$, al iterar hasta n y vaciar S en cada iteración, perdemos las transiciones que llevan a estados finales usando transiciones de leer más de una letra
- Notar que si no limitamos el tamaño de S , puede tener un tamaño de $|Q| \cdot \frac{n}{2}$ cuando $i = \frac{n}{2}$, lo que da la noción de que ahora el tamaño de S depende del tamaño de la palabra, lo que causaría que no se cumpla con la complejidad pedida. En efecto eso ocurre:

$$\begin{aligned}\mathcal{O}(|w| \cdot |\Delta| \cdot |S|) &= \mathcal{O}(|w| \cdot |\Delta| \cdot \sum_{i=1}^n (|Q| \cdot \min\{i, n-i\})) = \mathcal{O}(|w| \cdot |\mathcal{A}| \cdot \sum_{i=1}^n \min\{i, n-i\}) \\ &= \mathcal{O}(|w| \cdot |\mathcal{A}| \cdot \sum_{i=1}^{\lfloor \frac{n}{2} \rfloor} i) = \mathcal{O}(|w| \cdot |\mathcal{A}| \cdot n^2) = \mathcal{O}(|w|^3 \cdot |\mathcal{A}|)\end{aligned}$$

Se usa el *min* ya que no se leen mas de n letras, por lo que al pasar la mitad de letras leídas el conjunto posible pasa a estar acotado por i y n

- Para que la intersección del final tenga sentido, nos quedamos con solo los estados en vez de las tuplas, dando así al S_Q que se usa. El $j = n + 1$ no es necesario, ya que por como está construido el loop en el algoritmo solo se llegan a tuplas con ese valor para el segundo elemento. Notar que al no haber mas de n letras, no se leerá más, por lo que no se puede llegar a $n + 2$